Sistema de Wave Picking

Gestión y optimización de la preparación de pedidos mediante agrupamiento de órdenes y planificación de recorridos

Licenciatura en Ciencias de la Computación

Facultad de Ingeniería · Universidad Nacional de Cuyo

Proyecto integrador · Algoritmos y Estructuras de Datos II

Estudiantes: Bernhard Chiara, Noguera Alvaro, Parra Sofía, Rodriguez Luciana

Profesores: Jorge Guerra, Carlos Catania

Fecha de entrega: 11/11/2026

Índice

1. Introducción

   1.1 Propósito · 1.2 Alcance · 1.3 Definiciones

2. Descripción general

   2.1 Problema · 2.2 Usuarios · 2.3 Supuestos y restricciones

3. Requisitos

   3.1 Funciones implementadas · 3.2 Características técnicas

4. Diseño del sistema

   4.1 Arquitectura · 4.2 Modelo de datos · 4.3 Generación de waves

   4.4 Asignación de pickers · 4.5 Recorridos · 4.6 Simulación y persistencia

5. Despliegue

   5.1 Entorno · 5.2 Instalación y ejecución

6. Manual de usuario

   6.1 Inicio · 6.2 Escenarios · 6.3 Waves y simulación · 6.4 Archivos

7. Conclusiones y trabajo futuro

1. Introducción

1.1 Propósito

Este documento describe el Sistema de Wave Picking correspondiente al proyecto integrador de Algoritmos y Estructuras de Datos II. Su contenido se centra en la estructura del repositorio entregado, las entidades que modela y los mecanismos que utiliza para generar waves, asignar viajes, planificar recorridos y representar la preparación de pedidos.

La descripción se fundamenta en el código Python, los archivos de escenarios y el README incluidos en el ZIP del proyecto. Se documentan componentes presentes en esa entrega; no se incluyen funcionalidades propuestas para desarrollos posteriores como si formaran parte de la implementación.

1.2 Alcance del sistema

El programa representa un almacén de comercio electrónico por medio de una grilla, estanterías, pasillos, puertas y zonas de despacho. A partir de productos almacenados, pedidos con distintas prioridades y operarios simulados (pickers), genera agrupaciones de trabajo denominadas waves. Luego distribuye unidades de productos entre pickers, calcula los desplazamientos necesarios y actualiza el estado de la preparación en una simulación.

El código contempla escenarios generados automáticamente, edición de mapas, lectura de archivos JSON y persistencia del progreso de una partida. La visualización utiliza Pygame y está separada de los módulos de dominio y de simulación. El proyecto aborda operaciones internas del almacén; no implementa facturación, pagos, seguimiento de envíos fuera del almacén ni administración de cuentas de usuario.

1.3 Definiciones y acrónimos

Término	Significado en el proyecto
Warehouse	Almacén modelado con sus dimensiones, elementos físicos, productos, pedidos y pickers.
Pedido (Order)	Solicitud con identificador, prioridad, estado e ítems de productos y cantidades.
Wave	Agrupación de registros de producto–pedido, limitada por unidades y pedidos distintos.
Registro producto–pedido	Datos auxiliares del generador: producto, pedido de origen, prioridad y cantidad disponible.
Picker	Operario simulado que tiene posición y capacidad máxima de carga en kilogramos.
Viaje	Asignación concreta de productos a un picker y una zona de despacho.
Rack	Estantería con ubicación y dimensiones, que contiene productos.
Zona de despacho	Espacio de entrega asociado con una capacidad máxima acumulada de peso.
K-means++	Método de elección de centroides iniciales empleado por el agrupamiento K-means.
Dijkstra y A*	Algoritmos para encontrar caminos sobre los grafos de circulación.
Tick	Paso discreto de avance del estado de la simulación.
JSON	Formato utilizado para serializar escenarios y partidas.

2. Descripción general

2.1 Problema y justificación

En un almacén que prepara pedidos independientes, varios operarios pueden desplazarse reiteradamente por las mismas zonas. El sistema busca organizar parte de ese trabajo agrupando solicitudes que involucran productos próximos y, posteriormente, decidiendo qué puede recoger cada picker según su carga disponible y las rutas transitables.

La agrupación de productos y la planificación física no son equivalentes: dos ubicaciones pueden estar próximas en coordenadas, pero separadas por estanterías o por pasillos de circulación dirigida. Por eso el proyecto distingue el agrupamiento espacial de los productos, la asignación del trabajo y la búsqueda de caminos en el almacén.

2.2 Usuarios y roles

El usuario de la aplicación es quien prepara o carga escenarios y observa el funcionamiento de la simulación. El repositorio no contiene gestión de cuentas, autenticación ni permisos diferenciados. Los pickers son objetos del dominio cuyo movimiento y trabajo calcula el programa; no son usuarios que inicien sesión.

2.3 Supuestos y restricciones

- El espacio se representa en dos dimensiones (x, y). Los productos almacenados agregan la coordenada z correspondiente a la posición vertical dentro del rack.
- Las posiciones transitables y las direcciones permitidas forman un grafo dirigido. La navegación respeta la conectividad de ese grafo.
- Cada pedido tiene prioridad entera de 1 a 5: 5 es la más alta. Sus ítems indican producto y cantidad solicitada.
- El stock disponible se distribuye lógicamente al preparar los registros de las waves; el stock del producto se descuenta durante la recolección simulada.
- Los límites wave_max_pedidos y wave_max_productos deben ser enteros positivos. El valor 0 no significa capacidad ilimitada en esta entrega.
- La wave limita pedidos distintos y unidades. El peso se verifica al armar los viajes para cada picker y al reservar la capacidad de las zonas de despacho.
- Un pedido puede quedar repartido entre distintas waves. Una wave puede dar lugar a varios viajes y ser atendida por diferentes pickers.
- Si un producto no tiene un acceso válido hacia despacho, el simulador contempla su exclusión y la notificación correspondiente.

3. Requisitos

3.1 Funciones implementadas

La tabla resume operaciones presentes en los archivos del proyecto. En lugar de registrar funcionalidades hipotéticas, cada fila identifica el componente que contiene la implementación.

ID	Función	Archivo principal
RF-01	Modelar almacén, racks, pasillos, puertas y despachos.	domain/Warehouse.py
RF-02	Cargar productos, pedidos y pickers desde un escenario.	simulation/Cargador.py
RF-03	Generar escenarios y validar su estructura.	simulation/Scenario.py
RF-04	Consolidar solicitudes y respetar el stock disponible.	simulation/PreparacionRegistros.py
RF-05	Agrupar registros y construir waves sujetas a capacidad.	simulation/GeneradorWaves.py y módulos asociados
RF-06	Asignar viajes a pickers y despachos según peso y factibilidad.	simulation/AsignadorPickers.py
RF-07	Calcular y ordenar visitas con estrategias de recorrido.	simulation/PlanificadorViajes.py y Recorridos.py
RF-08	Buscar caminos con Dijkstra o A*.	simulation/Recorridos.py
RF-09	Gestionar movimientos y evitar colisiones entre pickers.	simulation/GestorColisiones.py
RF-10	Actualizar preparación, recolección y entrega durante la simulación.	simulation/Simulation.py
RF-11	Guardar y cargar escenarios y partidas en JSON.	simulation/Persistence.py
RF-12	Representar gráficamente el almacén y su ejecución.	visualization/App.py y VistaMapa.py
RF-13	Ofrecer generación y edición de escenarios en la interfaz.	visualization/GeneradorPantalla.py y EditorEsc.py

3.2 Características técnicas

Aspecto	Implementación que figura en el repositorio
Tecnología	Código en Python; interfaz de escritorio desarrollada con pygame-ce.
Separación de responsabilidades	Carpetas domain, simulation y visualization con módulos independientes.
Formato de datos	Escenarios y estados de partidas serializados como archivos JSON.
Reproducibilidad del generador	Generación de clusters con random.Random y semilla; ordenamientos con criterios explícitos.
Búsqueda acotada en asignación	Alternancia entre retroceso y estrategia voraz según un umbral de combinaciones.
Validación	Comprobaciones de capacidades, entidades, identificadores de productos y accesibilidad.

4. Diseño del sistema

4.1 Arquitectura y organización del código

La aplicación está organizada en tres paquetes principales dentro de proyeto_integrador. La capa domain define las entidades del problema; simulation implementa la generación de waves, la navegación, la asignación y la evolución del estado; visualization organiza la interfaz Pygame. La clase Sesion conecta la interfaz con la simulación sin trasladar los algoritmos al código visual.

Paquete / módulo	Responsabilidad en el sistema
domain/	Warehouse, Rack, Aisle, Door, DispatchZone, Product, Order, OrderItem, Picker, Wave y validaciones.
simulation/Cargador.py y Scenario.py	Construcción de objetos de dominio desde JSON, generación y validación de escenarios.
simulation/GeneradorWaves.py y colaboradores	Preparación de registros, cálculo de K, agrupamiento K-means, construcción y ordenamiento.
simulation/AsignadorPickers.py y PlanificadorViajes.py	Creación de viajes, elección de zonas de despacho y estimación de distancias.

Paquete / módulo	Responsabilidad en el sistema
simulation/Navegation.py y Recorridos.py	Grafo de circulación, búsqueda de caminos y orden de visita.
simulation/Simulation.py, EstadoPicker.py y GestorColisiones.py	Ejecución por ticks, estados de operarios, recogida, entrega y resolución de conflictos de movimiento.
simulation/Persistence.py y Sesion.py	Persistencia JSON y coordinación entre el estado del simulador y la interfaz.
visualization/	Pantallas del almacén, paneles, visualización, editor y generación de escenarios.

El punto de entrada de la aplicación es visualization/Main.py. A partir de los datos iniciales se construye una Sesion, que contiene una Simulacion. Si no se le proporciona una lista de waves, la simulación llama al generador integrado del proyecto.

4.2 Modelo de datos

Entidad	Información y comportamiento relevantes
Warehouse	Dimensiones, puertas, despachos, racks, pasillos, productos, pedidos, pickers y generación de grafos.
Product	ID, nombre, rack, ubicación (x, y, z), stock, peso_kg y acceso transitable.
Order / OrderItem	ID, prioridad y estado del pedido; producto y cantidad de cada ítem.
Wave	ID, pedidos asociados, registros products con order/product/quantity, límites y estado.
Picker / EstadoPicker	Datos físicos del picker y estado operativo usado durante la simulación.
DispatchZone	ID, puerta, ubicación, dimensiones y peso máximo admitido.
Aisle / Rack / Door	Geometría y atributos de los pasillos, estanterías y accesos.

En la representación de una wave, products es una lista de diccionarios que contienen las claves order, product y quantity. Esta estructura conserva el vínculo entre el producto solicitado y su pedido de origen. El asignador transforma luego esos registros en cantidades pendientes por producto y en trabajos individuales para los pickers.

Estructura del escenario JSON

El archivo domain/escenario_basico.json incluido en el ZIP permite reconocer las secciones de entrada que consume el cargador. La clave warehouse agrupa dimensiones, puertas, zonas_despacho, racks y pasillos; las listas pickers, productos y pedidos contienen las entidades de la operación; configuracion fija las capacidades por wave.

Sección	Campos utilizados en la carga
warehouse.dimensiones	ancho_x, largo_y
warehouse.puertas	id, tipo, ubicacion (x, y)
warehouse.zonas_despacho	id, puerta_id, ubicacion, ancho, alto, peso_max_kg
warehouse.racks / pasillos	id, ubicacion, ancho, alto; sentido para pasillos
pickers	id, posicion_inicial, capacidad_carga_kg
productos	id, nombre, rack_id, ubicacion (x, y, z), cantidad_disponible, peso_kg
pedidos	id, prioridad, estado, items de producto_id y cantidad
configuracion	wave_max_pedidos, wave_max_productos

4.3 Generación de waves

El procedimiento generar_waves(pedidos, max_pedidos, max_unidades, semilla=42), definido en simulation/GeneradorWaves.py, ejecuta cinco etapas. El generador agrupa solicitudes; no asigna pickers y no determina recorridos. Estos trabajos corresponden a componentes posteriores.

Preparación de registros

PreparacionRegistros.py recorre los pedidos y consolida las líneas repetidas del mismo producto dentro de cada uno. Comprueba que las asociaciones con Product coincidan y que las cantidades sean positivas. Después ordena las solicitudes por prioridad descendente, identificador de pedido y producto; distribuye el stock entre pedidos con un saldo lógico y conserva solamente la cantidad disponible. El stock real de Product no se modifica en esta etapa.

Cálculo de K inicial

CalculoK.py estima la cantidad inicial de clusters a partir del total de unidades, el número de pedidos distintos y los límites configurados. La fórmula implementada es:

```
k_unidades = ceil(total_unidades / max_unidades)
k_pedidos  = ceil(total_pedidos / max_pedidos)
K = min(max(k_unidades, k_pedidos), total_registros, ubicaciones_distintas)
```

El resultado K es una referencia para el agrupamiento: no determina la cantidad final de waves. El número final también depende de cómo se repartan los registros entre los clusters y de los límites aplicados después.

Agrupamiento espacial mediante K-means

KMeans.py utiliza la ubicación tridimensional (x, y, z) de los productos. Inicializa los centroides mediante K-means++, ponderando ubicaciones repetidas y las distancias cuadradas al centroide más próximo. En cada iteración asigna cada registro al centroide más cercano y recalcula la media de las coordenadas de cada cluster. El procedimiento se detiene si las asignaciones no cambian o si alcanza el máximo de 300 iteraciones; también elimina los clusters vacíos del resultado.

La inicialización usa random.Random(semilla), con valor predeterminado 42. Los clusters resultantes se entregan directamente al módulo encargado de construir las waves de acuerdo con los límites de capacidad.

Construcción y ordenamiento de waves

ConstruccionWaves.py procesa los registros de cada cluster según prioridad descendente y desempates por pedido y producto. Si una solicitud individual supera max_unidades, la divide en tramos que sí pueden entrar en una wave. Para cada wave incorpora registros que no excedan la cantidad máxima de unidades ni la de pedidos distintos; los que no entran permanecen pendientes para otra wave. No combina registros pertenecientes a clusters distintos dentro de una misma wave.

Por último, OrdenamientoWaves.py calcula para cada wave una tupla (P5, P4, P3, P2, P1), donde Pi cuenta registros asociados a pedidos de prioridad i, no unidades. La lista de waves se ordena en forma lexicográfica descendente; en los empates se preserva el orden relativo anterior.

4.4 Asignación de pickers y zonas de despacho

La clase Asignador, en simulation/AsignadorPickers.py, recibe las waves sin operario asignado. Normaliza los registros de la wave, consolida cantidades por producto y mantiene las unidades pendientes. En cada ronda considera los pickers libres y produce, como máximo, un viaje nuevo por picker. A medida que se entregan viajes, los operarios vuelven a quedar disponibles para recibir trabajo de waves pendientes.

Para elegir una asignación, el módulo expande las cantidades pendientes como una secuencia de unidades. Si hay P pickers disponibles y Q unidades por decidir, utiliza (P + 1)^Q como estimación del espacio de combinaciones, incluyendo la opción de no asignar una unidad en la ronda. Con el umbral predeterminado de 5000, utiliza búsqueda con retroceso cuando el tamaño estimado no supera ese valor; para tamaños mayores emplea una estrategia voraz.

El backtracking considera las capacidades en kilogramos de los pickers y las rutas factibles. Compara las asignaciones mediante un vector binario de decisiones, de manera lexicográfica, que favorece asignar las unidades ubicadas antes en la secuencia de evaluación. Cuando el vector coincide, compara los costos estimados de viaje y la elección de despachos. La búsqueda descarta ramas cuyo prefijo de decisiones no puede mejorar la candidata actual. La alternativa voraz evalúa opciones factibles para cada unidad y elige según el criterio de costo del momento.

En ambos casos, PlanificadorViajes.py estima las distancias antes de crear el trabajo definitivo. El asignador también lleva por zona el peso ya entregado y el peso reservado en viajes activos. Cuando una zona no tiene capacidad restante suficiente, deja de ser candidata para nuevos viajes. Si hay unidades imposibles de asignar, registra el motivo y puede señalar la wave como incompleta.

4.5 Navegación y planificación de recorridos

Warehouse.py construye la representación de circulación del almacén. Navegation.py mantiene los puntos transitables, los accesos de productos y los destinos de despacho; la implementación admite un grafo reducido para planificar tramos y un grafo completo para los desplazamientos que lo requieran. Las aristas expresan movimientos permitidos, por lo que la cercanía geométrica no equivale a un camino válido.

Recorridos.py implementa Dijkstra y A*; esta última búsqueda utiliza la distancia Manhattan como heurística. Ambos procedimientos devuelven la longitud del camino, la secuencia de posiciones y una cuenta de nodos expandidos. Cuando un origen o destino no forma parte del grafo se informa el error; cuando no existe camino, el resultado queda como inalcanzable.

El orden de visita se obtiene con tres estrategias: orden_pedidos respeta el orden recibido; cercania escoge de manera iterativa el próximo punto más próximo según el costo de la ruta; zonas agrupa accesos en función de la orientación geométrica de los racks. PlanificadorViajesV utiliza la combinación elegida de estrategia y algoritmo de camino para estimar el viaje desde el picker, pasando por los productos, hasta un punto de despacho. También almacena resultados de búsqueda y de grafos para reutilizarlos durante la planificación.

4.6 Simulación, estado y persistencia

Simulation.py coordina la ejecución por ticks. Carga el almacén desde los datos del escenario, construye la navegación, prepara las waves si no se recibieron explícitamente, crea el asignador y transforma los pickers del dominio en objetos EstadoPicker. Durante la ejecución los pickers se desplazan, recogen productos, transportan su carga a la zona asignada y liberan el viaje al entregar.

GestorColisiones.py interviene en la coordinación de movimientos para impedir que los pickers ocupen simultáneamente posiciones incompatibles. El simulador puede replantear desplazamientos ante bloqueos. Las waves conservan estados de trabajo como creada, en proceso, completada o incompleta según el progreso registrado por el asignador y la simulación.

Persistence.py distingue dos tipos de archivo. Un escenario guarda la estructura inicial y la configuración del almacén; una partida contiene el escenario y el estado serializado de la simulación, incluyendo tick, pickers y waves. Al cargar una partida se reconstruyen los viajes activos, el stock recogido y la ocupación de los despachos. Main.py intenta restaurar una partida de autoguardado al iniciarse sin un escenario indicado expresamente.

5. Despliegue

5.1 Requisitos del entorno

El repositorio contiene código Python y un archivo requirements.txt dentro de proyeto_integrador. Este declara pygame-ce==2.5.8 y setuptools==84.0.0. La interfaz gráfica requiere un entorno de escritorio compatible con Pygame; el README contempla Windows, Linux y macOS. No se configura una base de datos ni un servidor web, ya que los escenarios y partidas son archivos JSON locales.

5.2 Instalación y ejecución

Desde la raíz del repositorio se crea un entorno virtual y se instalan las dependencias declaradas. Ejemplo de comandos en Linux/macOS:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r proyeto_integrador/requirements.txt
```

Para ejecutar la interfaz desde la raíz:

```
python proyeto_integrador/visualization/Main.py
```

Para cargar un escenario determinado al iniciar:

```
python proyeto_integrador/visualization/Main.py --escenario
proyeto_integrador/domain/escenario_basico.json
```

En Windows, el README indica activar el entorno desde .venv\Scripts\Activate.ps1. Las rutas se deben ajustar al directorio desde el cual se ejecutan los comandos. El escenario de ejemplo se encuentra en domain/escenario_basico.json.

6. Manual de usuario

6.1 Inicio del sistema

La aplicación se inicia desde visualization/Main.py. Sin un argumento --escenario, busca primero el último autoguardado y, si no lo encuentra o no puede recuperarlo, genera un escenario inicial. Con --escenario se solicita la carga del archivo JSON indicado. Una vez construido el estado inicial, la aplicación abre la visualización Pygame del almacén.

En la vista se representan el mapa y sus elementos, la posición de los pickers y la información de waves y progreso. La coordinación entre interfaz y lógica corresponde a Sesion.py.

6.2 Preparación de escenarios

El repositorio permite obtener un escenario desde la generación automática de Scenario.py, utilizar el editor de mapa de EditorEsc.py o cargar un JSON existente. Los escenarios describen las

dimensiones, las entidades físicas, la configuración de los pickers, los productos disponibles, los pedidos y los límites de waves.

En la edición de mapas, el modelo valida la disposición de sus componentes antes de convertir el diseño en datos de escenario. Los productos se ubican en racks y deben tener un acceso transitable. Los datos importados también pasan por validaciones de estructura y accesibilidad.

6.3 Preparación de pedidos en la simulación

Al crear la simulación, los pedidos se convierten en registros disponibles y después en waves. El asignador selecciona los viajes que pueden realizar los pickers libres según capacidad y factibilidad del recorrido. A partir de esas asignaciones, la simulación actualiza posiciones, recogidas, entregas y estado de cada wave.

Las decisiones de recorrido utilizan Dijkstra o A*, junto con una estrategia de orden de visita definida en el planificador. La actividad de los pickers se refleja en la vista del almacén y en los estados obtenidos por el simulador. Si un producto resulta inaccesible o una cantidad no puede asignarse por las restricciones disponibles, el código registra esa situación en la simulación.

6.4 Almacenamiento y recuperación

Guardar un escenario produce un JSON con la configuración inicial. Guardar una partida produce otro JSON que incluye, además, el avance de la simulación. Son formatos distintos y el cargador verifica el tipo de archivo al abrirlo. La partida reanudada conserva, entre otros datos, la ubicación de los pickers, el trabajo activo, las waves pendientes y el estado de la capacidad de despacho.

El autoguardado utiliza una ubicación de estado local del sistema operativo: %LOCALAPPDATA% \WavePicking\ultima_partida.json en Windows; o $XDG_STATE_HOME/WavePicking/ultima_partida.json en Linux/macOS, con alternativa ~/.local/state/WavePicking/ultima_partida.json cuando esa variable no está definida.

7. Conclusiones y trabajo futuro

El código entregado reúne un modelo del almacén, generación de waves basada en K-means, asignación de unidades a pickers, búsqueda de caminos, planificación de viajes y una simulación gráfica con persistencia en JSON. La separación entre dominio, simulación y visualización permite identificar qué componente transforma las solicitudes, cuál decide la distribución del trabajo y cuál representa el progreso.

La solución utiliza procedimientos concretos y acotados: las waves se agrupan espacialmente y respetan límites positivos; los viajes consideran capacidades de peso y distancias sobre el grafo; la asignación recurre al backtracking o a una alternativa voraz según la dimensión del problema. Se trata de estrategias de planificación y no de una garantía de optimización global.

Como líneas posibles de evolución —no implementaciones presentes en el ZIP— se pueden considerar alternativas de agrupamiento que contemplen directamente los costos reales de circulación, así como mejoras para reducir el trabajo de evaluación de viajes en escenarios grandes. Estas posibilidades quedan expresadas como trabajo futuro y no como características disponibles.